Saga Pattern

Category: Data consistency / distributed transactions **One-liner:** Break a business transaction that spans several services into a sequence of local transactions, each with a compensating action that undoes it — achieving eventual consistency without **distributed locks** (locks held across several machines at once so no one else can change the data).

Part 0 — Concepts you need first

What is a transaction?

A **transaction** is a single logical operation made up of one or more data-manipulation steps, which **succeed or fail together as a unit**.

The classic example is a bank transfer: subtract £100 from account A, add £100 to account B. If only the first step runs, £100 has vanished. The two steps must be **one indivisible unit**.

What is ACID?

The four properties a traditional database transaction guarantees.

Atomicity — the transaction is treated as a single unit that either **succeeds completely or fails completely**. If any part fails, the whole thing fails and the database is left unchanged. *"Atomic" means indivisible — you can't have half of it.*

Consistency — the transaction brings the database from one **valid state to another**, respecting all its rules and constraints.

Isolation — **concurrent** transactions (ones running at the same time) leave the database in the same state as if they had run **one after another**. Your half-finished transaction is invisible to everyone else.

Durability — once **committed** (finalised and saved), it **stays committed**, even if the machine loses power a millisecond later.

Together: once a transaction completes, you can be confident it **either fully occurred or didn't occur at all**.

What is a "rollback"?

Undoing a transaction's changes, returning the database to its state before the transaction began. In a single database this is automatic and free — the database keeps a log and reverses it.

This is the capability you lose when the transaction spans services, and replacing it manually is what the Saga pattern is about.

What is a database lock?

While a transaction modifies a row, the database **locks** it so nobody else can modify it simultaneously. That's what makes isolation work.

The relevant consequence: **locks are held until the transaction commits.** A short transaction holds a lock for milliseconds. A transaction waiting on a network call to another continent holds it for seconds — during which nobody else can touch that row.

What is "database per service"?

A core **microservices** principle (microservices = an application built as many small, independently deployed services): **each service owns its own database**, and no other service reads or writes it

directly. If Order Service needs inventory data, it *asks* Inventory Service; it does not query Inventory's tables.

This gives independence — Inventory can change its **schema** (the structure of its tables and columns) freely — but it's precisely what removes the option of one database transaction spanning both.

What is eventual consistency?

A model where the system **may temporarily be in an inconsistent state**, but will **eventually converge** to a consistent one.

Strong consistency: the moment your transfer completes, every observer sees both accounts updated. Eventual consistency: for a brief window, one observer might see the debit but not yet the credit — though it will resolve shortly.

Distributed systems (systems spread across many machines talking over a network) trade strong for eventual consistency because strong consistency across a network requires locks and coordination that cost **availability** (being able to answer requests at all) and speed. See System Design Fundamentals for consistency models and the CAP trade-off.

Part 1 — The problem

It works fine in a monolith

An e-commerce checkout: adjust inventory, create the order, take payment. In a **monolithic** application (built and deployed as one unit) with a single database, this is one ACID transaction. If payment fails, the database rolls back inventory and the order automatically. **You get atomicity for free.**

Why it breaks in microservices

Coordination. Inventory Service, Order Service, and Payment Service run on different servers, each with its **own database**. Enforcing atomicity means undoing all previous steps when any step fails — and **coordinating that rollback across independent services is genuinely challenging**. There is no shared transaction log, no single database to tell "undo everything."

Network latency (the delay each network call adds). Services may be distributed across different geographic locations, so a transaction can take a significant amount of time. **Holding resources locked for that long is impractical** and causes performance problems.

> Made concrete: a database transaction normally holds locks for a few milliseconds. A distributed transaction across three services with network calls between them might hold them for **several seconds**. During that window, every other customer trying to buy the same product is blocked. At any real traffic level, the system grinds to a halt.

The conclusion: the traditional transaction model isn't designed for the complexity and demands of distributed systems. ACID's principles, well-intentioned as they are, **don't translate to a world where services operate independently and communicate over networks.**

Part 2 — The solution

A saga is a sequence of local transactions.

Each local transaction:

1. **Updates its own database** (a normal, fully ACID transaction — just local)
2. **Publishes an event** (sends out a message announcing what just happened, for any interested service to react to) that triggers the next local transaction in the saga

If a local transaction **fails because it violates a business rule** (a rule of the domain, such as "stock can't go below zero"), the saga executes a series of **compensating transactions** that undo the changes made by the preceding local transactions.

Rather than trying to enforce strict consistency as ACID does, Saga **acknowledges** that distributed transactions span multiple services and databases, and opts for the more practical **eventual consistency** model.

The guarantee it actually gives

Either **all transactions complete**, or **compensating transactions run to roll back** the ones already executed. The system maintains integrity through explicit compensation rather than automatic rollback.

This makes sagas ideal for **long-lived, high-level business transactions spanning multiple microservices**, across globally distributed and **highly available** (designed to stay up nearly all the time) databases.

The cost: designing sagas is complex. They require intricate coordination between participating services and **extensive error handling**. You are hand-writing what the database used to do for you.

The metaphor worth keeping

> If your distributed system is a city and each service is a building, **ACID transactions are the internal wiring of each building** — ensuring electricity flows where needed inside. But when you want to connect two buildings, you can't just extend the internal wiring; it wouldn't be safe or efficient. **You need a power line.** The Saga Pattern is that power line — connecting the buildings so they work together, while maintaining safety and consistency.

Part 3 — The two types

Choreography — decentralized

Every local transaction **publishes domain events** (messages describing a business fact, like `OrderCreated`) that trigger local transactions in other services. **There is no central coordination.** Each service produces events and listens to other services' events, deciding for itself whether to act.

> An orchestra with **no conductor** — each musician listens to the other instruments and plays their part in harmony. Or an ant colony, where each ant knows its tasks and performs them without being told.

What it looks like in practice: Order Service publishes `OrderCreated`. Payment Service is listening, sees it, charges the customer, publishes `PaymentCompleted`. Inventory Service is listening for that, reserves stock, publishes `StockReserved`. Nobody is in charge; the flow emerges from who **subscribes** to (registers interest in) what.

Orchestration — centralized

A central **saga orchestrator** is responsible for executing the saga's steps. It tells each participant when to execute its local transaction.

> The same orchestra, **with** a conductor guiding all the musicians to create a symphony. Or a busy airport where an air traffic controller tells each plane when to take off and land.

What it looks like in practice: the orchestrator calls Order Service, waits, calls Payment Service, waits, calls Inventory Service. If Inventory fails, the orchestrator calls the compensations in reverse. The whole flow is readable in one place.

Choosing between them

	Choreography	Orchestration
Coordination	None — fully decentralized	Central orchestrator
Service autonomy	Higher	Lower — services do as told
Visibility of the flow	Poor — scattered across services	Good — in one place
Error handling	Harder	Easier
Coupling (how much one part depends on another)	Services coupled to events	Services coupled to the orchestrator
Best for	**Simple** transactions	**Complex** transactions

The rule of thumb the course gives: consider the complexity of your business transaction. If it's simple, choreography is a good option as it requires less coordination. As it becomes more complex, **orchestration provides better visibility and control**.

> The practical failure mode of choreography: with six services each reacting to events, **nobody can answer "what happens when a customer places an order?"** without reading six **codebases** (separate bodies of source code). The business process exists but is written nowhere. That's why complex flows migrate to orchestration.

Part 4 — Architecture

Local transactions — the backbone

A saga is composed of multiple local transactions, each executed **within a single service and its corresponding database**. Each carries its own responsibilities, but strung together they work toward maintaining data consistency across the system.

Every local transaction has an associated compensating transaction capable of undoing its changes.

Compensating transactions

If a local transaction fails, the saga executes compensating transactions for the previously successful ones, bringing the system back to a consistent state.

> Walking along a path and realising you've gone the wrong way — you retrace your steps back to

> where you took the wrong turn.

The Saga Log

To track the saga's progress and state, a **saga log** is maintained. It records **every local transaction and compensating transaction executed** in the saga.

Its purpose: if the saga is interrupted — a service crashes mid-flow — the log lets it **resume from where it left off**, or know exactly which compensations to run.

> Hansel and Gretel's breadcrumb trail — if you get lost (a system failure), follow the breadcrumbs (the saga log) back home (a consistent state).

Database per service

Each service **owns its database**, ensuring the failure of one doesn't directly impact others. This prevents a **single point of failure** (one component whose failure brings everything down) and makes the system more robust and reliable.

> A team of rock climbers, each with their own safety rope. If one slips, their rope catches them without pulling the rest down.

Eventual consistency

The ultimate goal. The system **may pass through inconsistent states**, but will **eventually reach a consistent one**.

> Juggling: at any given moment some balls are in the air, but a skilled juggler ensures all get handled correctly without dropping any.

Part 5 — Worked example: online shopping (orchestration)

The scenario: you add items to your cart and check out, creating an order. The Inventory Service checks whether items are available. If yes, it updates inventory and the order completes. If not, the order is cancelled.

The services — each with its own database

```
public class OrderService {
    private OrderDatabase orderDatabase = new OrderDatabase();

    public void createOrder(Order order) {
        orderDatabase.save(order);
    }

    public void cancelOrder(String orderId) {
        // COMPENSATING TRANSACTION for createOrder
    }
}
```

```
public class InventoryService {
    private InventoryDatabase inventoryDatabase = new InventoryDatabase();

    public boolean checkAndReduceInventory(Order order) {
        // local transaction — returns false if stock is insufficient
    }

    public void addBackInventory(Order order) {
        // COMPENSATING TRANSACTION for checkAndReduceInventory
    }
}
```

Notice the structure — this is the pattern in miniature. Each service has its **own database** (`OrderDatabase`, `InventoryDatabase`), and each forward operation is **paired with an inverse**:

Forward action	Compensating action
`createOrder`	`cancelOrder`
`checkAndReduceInventory`	`addBackInventory`

That pairing *is* the Saga pattern. Everything else is plumbing to invoke them in the right order.

The orchestrator

```
public class OrderOrchestrator {
    private OrderService orderService = new OrderService();
    private InventoryService inventoryService = new InventoryService();

    public void processOrder(Order order) {
        orderService.createOrder(order);

        if (!inventoryService.checkAndReduceInventory(order)) {
            orderService.cancelOrder(order.getId());   // compensate
        }
    }
}
```

Reading it: create the order first. Then attempt to check and reduce inventory. If that returns `false` — insufficient stock — **undo the order** by cancelling it.

The forward path and the compensation path are both visible in one method. That readability is orchestration's main selling point.

Handling mid-saga failures

The course raises the obvious question: *what if a service fails partway through?*

That's what the **Saga Log** is for. You expand the services to record each step. If a service fails, consult the log to see where it left off and execute the necessary compensating transactions.

> Notice this example has **no saga log at all**. If the process crashes between `createOrder` and `checkAndReduceInventory`, the order exists forever in a limbo state with no inventory reserved and nothing to fix it. That's why real implementations persist saga state before every step.

What the example is and isn't

It's a basic illustration. A real application involves more services, more complexity, and more failure modes — no retries, no **asynchronous messaging** (sending messages without waiting for a

reply), no log here. But the value of the pattern holds: it lets you **split a large transaction into manageable, independent parts that can be executed and compensated for separately.**

Part 6 — Issues and considerations

Complexity

Splitting a large transaction into several smaller ones ensures consistency but **adds complexity**. Each service must now handle **both** its local transaction **and** provide a compensating transaction.

The course's concrete difficulty: what happens if a **discount was applied** to the order, and the order then gets cancelled because items were unavailable? The Order Service's compensating transaction must handle unwinding the discount too — perhaps returning a single-use voucher to the customer's account. Complexity creeps in quickly.

The pattern requires **careful design and implementation to avoid creating a tangled web of services.**

Longer latency

In a distributed transaction, **each service call is a network call**, and network calls are slower than local ones. Sagas therefore have **longer latency than traditional transactions**.

In a monolith, placing an order and updating inventory happens quickly inside a single database transaction. With a saga, these operations happen over the network, introducing delay.

Application consistency — not just data consistency

This is a subtle point and the most important one in the chapter.

Saga ensures **data** consistency by executing compensating transactions. But what about the **business process**?

> If the inventory check fails, the order gets cancelled. **But from a business perspective, cancelling an order is not the same as never having placed it.**

The customer may have already received a confirmation email. There's an **audit record** (a permanent log entry kept for accountability) showing an order existed. **Analytics** (the reporting system) counted it. A warehouse worker may have seen it on a screen.

Compensation is not time travel. It gets the data back to a consistent state, but it cannot un-happen the visible intermediate state. These considerations must be thought through when designing the saga.

Idempotency

An operation could **fail after executing but before reporting success**. In that case it may be retried — so the operations must be **idempotent** (performing them multiple times yields the same result as once).

The example: if `checkAndReduceInventory` reduces inventory but fails before reporting success, a retry must **not reduce the inventory a second time**. So that method needs to be idempotent — typically by recording the **saga ID** (a unique identifier for one run of the saga) and refusing to apply the same operation twice. *(See Retry for idempotency keys.)*

Increased load

Each service must maintain a **Saga Log** to track the saga. That means **extra read and write operations for every service**, increasing **load** (the amount of work the service must handle) and potentially

slowing response times. The Order Service now handles orders *and* manages its saga log.

It's a trade-off

As with most things in software architecture, this is about balance. Would you choose consistency over complexity, or speed over distributed transactions? Understanding the trade-offs is what lets you decide well.

Part 7 — Use cases

E-commerce systems

Order placement is a textbook distributed transaction. An **Order service** creates the order, a **Payment service** charges the customer, an **Inventory service** checks and reduces stock.

If the inventory check fails, the order must be cancelled **and the customer's payment refunded**. That refund is a compensating transaction — and note it's not a rollback. The charge really happened and really appears on the customer's statement; the refund is a second, separate movement of money.

Travel booking systems

Booking a trip involves reserving a flight, booking a hotel, and perhaps renting a car. These need to **succeed together or fail as a whole**. But they are **independent services, possibly owned by different companies**.

> This is the clearest case for Saga. There is no shared database — you cannot possibly run one ACID transaction across an airline, a hotel chain, and a car rental firm. Compensation (cancel the flight, cancel the hotel) is the *only* available mechanism.

Part 8 — System design examples

Online food delivery

When a user places an order, several steps are initiated: the order is created, the restaurant confirms it, a delivery partner is assigned, and payment is processed.

But what if the restaurant runs out of an item? The order needs to be cancelled and the payment refunded. Saga maintains data consistency across services, handles the failure scenario gracefully, and keeps the user experience seamless.

Internet banking

A user transfers money from savings to checking. The transaction involves two operations — **debiting savings** and **crediting checking**. They must happen in unison; if one fails, the other must not proceed.

> Without the Saga Pattern, a failure in the crediting operation could lead to **money disappearing into thin air** — debited from one account and never arriving in the other.

Part 9 — Saga vs. two-phase commit (2PC)

Not in the course, but you'll be asked to compare them.

2PC is the traditional way to make a transaction atomic across multiple databases. A **coordinator** (the component running the protocol) asks every participant "can you commit?" (phase 1), and if all say yes, tells them all "commit now" (phase 2).

	2PC	Saga
Consistency	Strong (genuinely ACID)	Eventual
Locks	Holds locks across all participants until commit	No distributed locks
Availability	**Blocks** if the coordinator fails mid-protocol	Stays available
Rollback	Automatic	Manual — you write the compensating logic
Suits	Short transactions in one trust domain (systems run by one organisation that trust each other)	Long-lived, cross-service, cross-company

Why microservices generally avoid 2PC: it holds locks for the duration of the whole distributed operation (the very problem described in Part 1), and if the coordinator dies between phases, participants are left holding locks with no instruction — a genuinely stuck system. It also requires every participant to support the protocol, which external companies won't.

Part 10 — Common beginner misconceptions

"A compensating transaction is a rollback." No. A rollback erases history; a compensation is a **new forward action that offsets the previous one**. The refund appears on the statement alongside the charge. Both are real.

"Every step can be compensated." Some can't. You cannot un-send an email, un-ship a package, or un-launch a rocket. For those, either put them **last** in the saga, or design a real-world compensation (a "please ignore that email" follow-up).

"Sagas give you ACID across services." They explicitly don't. You give up isolation entirely — other transactions **can see** your half-finished saga. That's the price of avoiding distributed locks.

"Choreography is simpler, so always start there." It's simpler for two or three services. Beyond that the invisible business process becomes a serious maintenance problem.

"The orchestrator is a single point of failure, so choreography is safer." The orchestrator can be made highly available and its state persisted. Choreography's risk isn't availability — it's comprehensibility.

"Eventual consistency means the data might just be wrong." No — it means there's a *window* during which observers may see intermediate state. It converges. The design question is whether your business can tolerate that window.

Part 11 — Check your understanding

1. Why can't you use one ACID transaction across three microservices?
2. Why is holding database locks across a distributed transaction impractical?
3. What is the pairing that defines the Saga pattern?

4. Why is "cancelling an order is not the same as never placing it" an important point?
5. Why must saga steps be idempotent?
6. Why does travel booking make the strongest case for Saga?

*(Answers: 1 — each service owns its own database, so there's no single **transaction manager** (the component that coordinates commit and rollback) or shared log to coordinate commit and rollback; 2 — the transaction spans network calls taking seconds, during which locked rows block every other user; 3 — each forward local transaction paired with a compensating transaction that undoes it; 4 — Saga restores data consistency but cannot undo visible side effects like confirmation emails and audit records, so the business process must account for the intermediate state; 5 — a step can succeed but fail before reporting success, then be retried, and must not apply its effect twice; 6 — the participants are separate companies, so a shared database transaction is not merely difficult but impossible.)*

Part 12 — Interview soundbite (30 seconds)

> "Saga solves distributed transactions when ACID can't cross service boundaries. Each service owns its own database, so there's no shared transaction manager to coordinate a commit — and holding locks across network calls for seconds would block every other user anyway. Instead you break the business transaction into a sequence of local transactions, each paired with a compensating transaction that undoes it. If step three fails, you run the compensations for steps two and one. You get eventual consistency rather than immediate consistency, and you give up isolation completely — other transactions can see your half-finished saga. There are two flavours: choreography, where each service emits events others react to, which is decentralized and fine for simple flows but scatters the business process across six codebases so nobody can read it; and orchestration, where a central coordinator drives the steps, giving much better visibility and error handling for complex flows. A saga log persists progress so you can resume or compensate after a crash. The classic examples are e-commerce checkout — order, payment, inventory, with a refund as compensation — and travel booking across flight, hotel, and car from different companies, where a shared transaction is simply impossible. The hard parts are that steps must be idempotent, since a step can succeed but fail to report success and then get retried, and that compensation isn't rollback: the refund is a second real movement of money, and cancelling an order isn't the same as never placing it, because the confirmation email already went out."

Part 13 — Implementation

A. Runnable demo

Full source: `demos/SagaDemo.java`

```
cd demos && ./gradlew runCh10
```

Compensation driven by the log, in reverse order:

```
void compensate(String sagaId) {
    for (String step : log.stepsNeedingCompensation(sagaId)) {   // newest first
        switch (step) {
            case "reserveStock"  -> inventory.release(sagaId);
            case "chargePayment" -> payments.refund(sagaId);
            case "createOrder"   -> orders.cancelOrder(sagaId);
        }
        log.record(sagaId, step, StepState.COMPENSATED);
    }
}
```

Idempotency using the saga ID as the key:

```
void charge(String sagaId, int amount) {
    if (charges.containsKey(sagaId)) {
        return;                       // duplicate - do NOT charge again
    }
    charges.put(sagaId, amount);
    ledger.add("CHARGE  +" + amount);
}
```

B. What you should see

```
=== 2. Step 3 fails -> compensations run in REVERSE ===
      OrderService: order created
      PaymentService: charged 500
      InventoryService: INSUFFICIENT STOCK (have 3, need 99)
    !!! step failed: out of stock -> compensating
      PaymentService: REFUNDED 500 (compensation)
      OrderService: order CANCELLED (compensation)
  order status: CANCELLED | stock: 3

=== 3. Idempotency: the same step retried ===
      PaymentService: charged 150
      PaymentService: duplicate detected, NOT charging again
      PaymentService: duplicate detected, NOT charging again

=== 4. Compensation is NOT rollback ===
  payment ledger: [CHARGE +300, CHARGE +500, REFUND -500, CHARGE +150]
```

Section 4 is the whole conceptual point, made concrete. Look at the **ledger** (an append-only list of money movements): the £500 charge and the £500 refund **both exist**. A database rollback would have erased the charge as though it never happened. Compensation adds an **offsetting entry** — and the customer sees both lines on their statement. That's why "cancelling an order is not the same as never placing it."

C. Production — orchestration

Hand-rolling saga orchestration is possible but you'll reinvent **persistence** (saving state so it survives crashes), **timeouts** (limits on how long to wait), and retries. Real options — all **workflow engines** that run multi-step processes reliably: **Temporal**, **Camunda/Zeebe**, **AWS Step Functions** (Amazon's managed version), or **Axon Framework** (a Java framework for event-based systems).

Temporal is the most popular code-first choice. Your saga is ordinary code; the framework makes it **durable** (its progress is saved and survives crashes) — if the process crashes mid-saga, it resumes from the last completed step on another **worker** (a process that executes workflow steps):

```java
public class OrderWorkflowImpl implements OrderWorkflow {

    private final OrderActivities activities = Workflow.newActivityStub(
        OrderActivities.class,
        ActivityOptions.newBuilder()
            .setStartToCloseTimeout(Duration.ofSeconds(10))
            .setRetryOptions(RetryOptions.newBuilder()
                .setMaximumAttempts(3)
                .setBackoffCoefficient(2.0)
                .build())
            .build());

    @Override
    public void processOrder(Order order) {
        Saga saga = new Saga(new Saga.Options.Builder().build());
        try {
            activities.createOrder(order);
            saga.addCompensation(activities::cancelOrder, order.id());   // register the undo

            activities.chargePayment(order);
            saga.addCompensation(activities::refundPayment, order.id());

            activities.reserveStock(order);                              // may fail
        } catch (ActivityFailure e) {
            saga.compensate();      // runs registered compensations in reverse
            throw e;
        }
    }
}
```

The pattern to copy even without Temporal: register the compensation **immediately after** each step succeeds. That way the list of things to undo is always exactly correct, with no bookkeeping.

Choreography with Kafka — no orchestrator; services react to each other's events. (**Kafka** is a distributed event log: producers write messages to named **topics** and **consumers** read them; a `groupId` names a group of consumers that share the work.)

```java
@KafkaListener(topics = "order-created", groupId = "payment-service")
public void onOrderCreated(OrderCreated event) {
    try {
        paymentService.charge(event.orderId(), event.amount());
        kafka.send("payment-completed", new PaymentCompleted(event.orderId()));
    } catch (PaymentFailedException e) {
        kafka.send("payment-failed", new PaymentFailed(event.orderId()));   // triggers compensation
    }
}
```

Note the shape: **every step publishes either success or failure**, and compensation is just another consumer reacting to a failure event. Nobody is in charge — which is exactly why complex flows become hard to follow.

D. Implementing the saga log properly

The demo's log is in memory, so a crash loses everything. Real implementations persist it **in the same transaction as the business change**, using the **Transactional Outbox** pattern (write outgoing events into an "outbox" table in the same database transaction as the data change, and publish them afterwards). In the SQL below, a **UUID** is a randomly generated unique ID, the **primary key** is the column that uniquely identifies each row, and `JSONB` is PostgreSQL's binary JSON column type:

```sql
CREATE TABLE saga_state (
    saga_id       UUID PRIMARY KEY,
    current_step  VARCHAR(50)  NOT NULL,
    state         VARCHAR(20)  NOT NULL,   -- STARTED / COMPLETED / COMPENSATING / DONE
    payload       JSONB        NOT NULL,
    updated_at    TIMESTAMP    NOT NULL
);

CREATE TABLE outbox (
    id          UUID PRIMARY KEY,
    aggregate   VARCHAR(50),
    event_type  VARCHAR(100),
    payload     JSONB,
    published   BOOLEAN DEFAULT FALSE
);
```

```java
@Transactional
public void completeStep(String sagaId, String step, Event next) {
    sagaRepo.markCompleted(sagaId, step);      // both writes
    outboxRepo.save(next);                     // in ONE local transaction
}
// A separate poller (or Debezium CDC) publishes unpublished outbox rows to Kafka
```

(In the code above, `@Transactional` makes the method run inside one database transaction; a **poller** is a process that checks the table repeatedly for new rows; **Debezium CDC** — change data capture — instead streams new rows straight from the database's change log.)

Why the outbox exists. If you write to the database and then publish to Kafka as two separate operations, a crash between them leaves the two permanently inconsistent — the step completed but nobody was told. Writing the event to the same database in the same transaction makes them atomic; a separate process then relays it. This is the standard solution and worth knowing by name.

E. Implementation pitfalls

1. Compensations that aren't idempotent. Compensation gets retried too. Refunding twice is as bad as charging twice. Guard with the saga ID.

2. Compensations that can fail permanently. What if the refund API is down? You need retries with **backoff** (waiting longer between each attempt), and ultimately a **dead-letter queue plus human intervention** (a dead-letter queue is a holding area for messages that keep failing, so a person can inspect them). A saga stuck half-compensated is the worst possible state — plan for alerting on it.

3. Putting an uncompensatable step in the middle. You cannot un-send an email or un-ship a parcel. Order your steps so **irreversible actions come last**, after everything that could fail has succeeded.

4. An in-memory saga log. As in the demo. A crash mid-saga leaves **orphaned** state (half-finished records that nothing owns or will clean up) with nothing to recover from. Persist before each step.

5. Writing to the database and publishing separately. Use the outbox. This is the single most common correctness bug in event-driven sagas.

6. Assuming isolation. Sagas have **no isolation** — other transactions see your intermediate state. A customer may see an order that's about to be cancelled. Design the **UI** (user interface) for it ("processing" rather than "confirmed").

7. Choreography for a complex flow. Six services reacting to each other's events means nobody can answer "what happens when an order is placed?" without reading six codebases. Move to orchestration

once you pass three or four steps.

8. No timeouts on saga steps. A step that never responds leaves the saga hanging forever. Every step needs a deadline, after which it's treated as failed and compensation begins.

Related patterns

- **Event-Driven Architecture** — the messaging substrate choreography sagas run on
- **CQRS** — frequently paired with sagas in **event-sourced** systems (which store every change as an event rather than just the current state)
- **Retry** — idempotency concerns are shared; retries happen within saga steps
- **Circuit Breaker** — protects individual saga participants